INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	**INTRODUCCIÓN AL DESARROLLO WEB**				
TITULO DE LA PRÁCTICA:	*Desarrollo Backend con Express.js, Ruteo, Middlewares y Arquitectura por Capas*				
NÚMERO DE PRÁCTICA:	*06*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*02*
FECHA DE PRESENTACIÓN	*26/10/2026*	**HORA DE PRESENTACIÓN**	*hh/mm/ss*		
INTEGRANTE (s) - Gomez Huaman, Melany Maritza - Morales Yapu, Hillary Fernanda - Ortiz Saavedra, Sarah Megan - Paredes Gonzales, Melanie Grace				**NOTA (0-20)**	
DOCENTE(s): **MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.**					

RESULTADOS Y PRUEBAS

I. EJERCICIOS RESUELTOS:

Estructura modular en Express por capas demostrada por el docente

```
// controllers/productController.js
let products = [
    { id: 1, nombre: 'Laptop', precio: 3500 }
];

exports.getAllProducts = (req, res) => {
    res.status(200).json({ status: 'success', data: products });
};

exports.createProduct = (req, res) => {
```

```
    const { nombre, precio } = req.body;
    if (!nombre || !precio) {
        return res.status(400).json({ status: 'fail', message: 'Faltan campos
obligatorios' });
    }
    const newProduct = { id: products.length + 1, nombre, precio:
parseFloat(precio) };
    products.push(newProduct);
    res.status(201).json({ status: 'success', data: newProduct });
};

// routes/productRoutes.js
const express = require('express');
const router = express.Router();
const productController = require('../controllers/productController');

router.get('/', productController.getAllProducts);
router.post('/', productController.createProduct);

module.exports = router;
// app.js
const express = require('express');
const productRoutes = require('./routes/productRoutes');

const app = express();
app.use(express.json()); // Middleware incorporado JSON parser

// Middleware propio Logger
app.use((req, res, next) => {
    console.log(`[${new Date().toISOString()}] ${req.method} en ${req.url}`);
    next();
});

app.use('/api/v1/products', productRoutes);

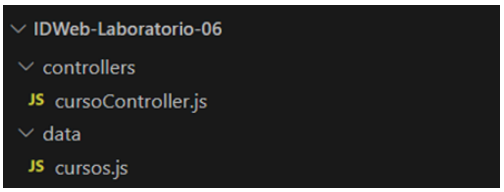
app.listen(3000, () => console.log('API Express en puerto 3000'));
```

II. PROBLEMAS PROPUESTOS RESUELTOS:

Caso Práctico: API RESTful para la Gestión de Cursos Académicos Construir un módulo backend con Express.js para administrar el recurso "Cursos", cumpliendo con la siguiente arquitectura y endpoints:

1. Arquitectura por Capas:

○ Separar estrictamente el código en las carpetas: /routes (definición de endpoints), /controllers (lógica de negocio) y /data (persistencia temporal en memoria o archivo JSON).





```javascript
const cursos = require('../data/cursos');

// Lecturas
const getAllCursos = (req, res) => {
    const { creditos } = req.query;
    let resultado = cursos;
    if (creditos !== undefined) {
        resultado = cursos.filter(c => c.creditos === Number(creditos));
    }
    res.status(200).json({ status: "success", results: resultado.length, data: resultado });
};

const getCursoById = (req, res) => {
    const id = parseInt(req.params.id);
    const curso = cursos.find(c => c.id === id);
    if (!curso) {
        return res.status(404).json({ status: "fail", message: "Curso no encontrado" });
    }
    res.status(200).json({ status: "success", data: curso });
};

// Escrituras
const createCurso = (req, res) => {
    const { nombre, codigo, creditos } = req.body;

    if (!nombre || !codigo || creditos === undefined) {
        return res.status(400).json({
            status: 'fail',
            message: 'Faltan campos obligatorios: nombre, codigo, creditos'
        });
    }

    const creditosNum = Number(creditos);
    if (Number.isNaN(creditosNum) || creditosNum <= 0) {
        return res.status(400).json({
            status: 'fail',
            message: 'creditos debe ser un número mayor que 0'
        });
    }
```

```
    const nuevoId = cursos.length > 0 ? Math.max(...cursos.map(c => c.id)) + 1 : 1;
    const nuevoCurso = { id: nuevoId, nombre, codigo, creditos: creditosNum };

    cursos.push(nuevoCurso);
    res.status(201).json({ status: 'success', data: nuevoCurso });
};

const updateCurso = (req, res) => {
    const id = Number(req.params.id);
    const curso = cursos.find(c => c.id === id);

    if (!curso) {
        return res.status(404).json({
            status: 'fail',
            message: `No se encontró el curso con id ${id}`
        });
    }

    const { nombre, codigo, creditos } = req.body;
    if (nombre !== undefined) curso.nombre = nombre;
    if (codigo !== undefined) curso.codigo = codigo;
    if (creditos !== undefined) curso.creditos = Number(creditos);

    res.status(200).json({ status: 'success', data: curso });
};

// Eliminación
const deleteCurso = (req, res) => {
    const id = Number(req.params.id);
    const indice = cursos.findIndex(c => c.id === id);

    if (indice === -1) {
        return res.status(404).json({
            status: 'fail',
            message: `No se encontró el curso con id ${id}`
        });
    }

    const [eliminado] = cursos.splice(indice, 1);
    res.status(200).json({
        status: 'success',
        message: `Curso ${eliminado.codigo} eliminado correctamente`
    });
};

module.exports = {
    getAllCursos,
    getCursoById,
    createCurso,
    updateCurso,
    deleteCurso
};
```

2. Middlewares Obligatorios:

○ Middleware global incorporado: express.json().
○ Middleware personalizado de Auditoría (Logger): Imprimir en consola el método, la URL y el tiempo de respuesta.

○ Middleware de Manejo de Errores: Capturar errores no controlados y responder un JSON formateado con código HTTP 500.

middlewares > JS errorHandler.js > ...

```
module.exports = (err, req, res, next) => {
  console.error(err.stack);

  // JSON mal formado enviado por el cliente
  if (err.type === 'entity.parse.failed') {
    return res.status(400).json({ status: 'fail', message: 'JSON mal formado' });
  }

  res.status(500).json({ status: 'error', message: 'Error interno del servidor' });
};
```

middlewares > JS logger.js > ...

```
module.exports = (req, res, next) => {
  const inicio = Date.now();

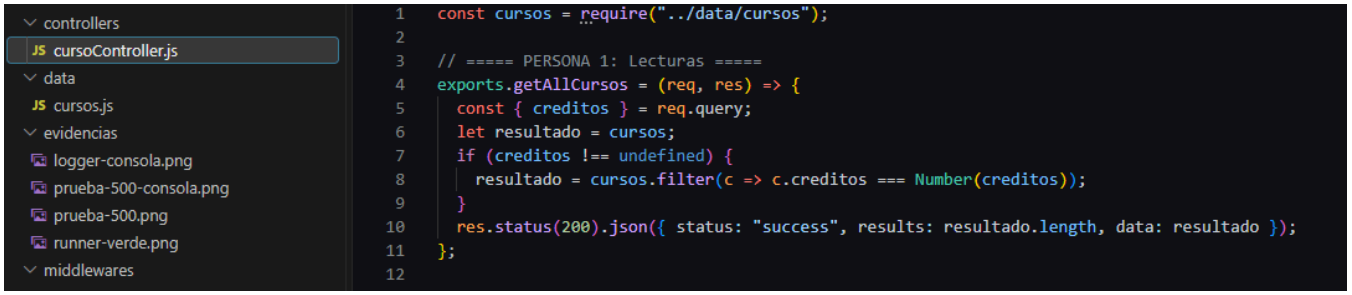
  res.on('finish', () => {
    const ms = Date.now() - inicio;
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl} - ${res.statusCode} - ${ms} ms`);
  });

  next();
};
```

3. Endpoints de la API REST (Recurso Cursos):

○ GET /api/cursos: Retorna lista completa de cursos con código 200. Permite filtrar mediante query string (ej. /api/cursos?creditos=4).

En controllers/cursoController.js:



Ejecución:

```
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> Invoke-RestMethod http://localhost:3000/api/cursos

status  results data
------  ------- ----
success       2 {@{id=1; nombre=Introducción al Desarrollo Web; codigo=IDWeb; creditos=4}, @{id=3; nombre=Estructuras Discretas II; .


PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> Invoke-RestMethod "http://localhost:3000/api/cursos?creditos=4"

status  results data
------  ------- ----
success       1 {@{id=1; nombre=Introducción al Desarrollo Web; codigo=IDWeb; creditos=4}}
```

```
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> (Invoke-WebRequest -UseBasicParsing "http://localhost:3000/api/cursos?credito
=4").StatusCode
200
```

○ GET /api/cursos/:id: Busca un curso por id. Retorna 200 si existe o 404 si no se encuentra.

En controllers/cursoController.js:

```
13  exports.getCursoById = (req, res) => {
14    const id = parseInt(req.params.id);
15    const curso = cursos.find(c => c.id === id);
16    if (!curso) {
17      return res.status(404).json({ status: "fail", message: "Curso no encontrado" });
18    }
19    res.status(200).json({ status: "success", data: curso });
20  };
```

Ejecucion:

```
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> Invoke-RestMethod http://localhost:3000/api/cursos/1

status  data
------  ----
success @{id=1; nombre=Introducción al Desarrollo Web; codigo=IDWeb; creditos=4}

PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> (Invoke-WebRequest -UseBasicParsing http://localhost:3000/api/cursos/1).Status
Code
200
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> Invoke-RestMethod http://localhost:3000/api/cursos/999
Invoke-RestMethod : {"status":"fail","message":"Curso no encontrado"}
En línea: 1 Carácter: 1
+ Invoke-RestMethod http://localhost:3000/api/cursos/999
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> try { Invoke-WebRequest -UseBasicParsing http://localhost:3000/api/cursos/999
} catch { $_.Exception.Response.StatusCode.value__ }
404
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06>
```

○ POST /api/cursos: Valida los campos obligatorios (nombre, codigo, creditos). Crea el curso y retorna 201 Created.

```
22  // ===== PERSONA 2: Escrituras =====
23  exports.createCurso = (req, res) => {
24    const { nombre, codigo, creditos } = req.body;
25    if (!nombre || !codigo || !creditos) {
26      return res.status(400).json({
27        status: "fail",
28        message: "Faltan campos obligatorios (nombre, codigo, creditos)"
29      });
30    }
31    const nuevoId = cursos.length ? Math.max(...cursos.map(c => c.id)) + 1 : 1;
32    const nuevo = { id: nuevoId, nombre, codigo, creditos: Number(creditos) };
33    cursos.push(nuevo);
34    res.status(201).json({ status: "success", data: nuevo });
35  };
36
```

Ejecucion:

```
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> Invoke-RestMethod -Method POST -Uri http://localhost:3000/api/cursos -ContentT
ype "application/json" -Body '{"nombre":"Redes","codigo":"RED-301","creditos":3}'

status  data
------  ----
success @{id=4; nombre=Redes; codigo=RED-301; creditos=3}


PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> (Invoke-WebRequest -UseBasicParsing -Method POST -Uri http://localhost:3000/ap
i/cursos -ContentType "application/json" -Body '{"nombre":"Redes","codigo":"RED-301","creditos":3}').StatusCode
201
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> Invoke-RestMethod -Method POST -Uri http://localhost:3000/api/cursos -ContentT
ype "application/json" -Body '{"nombre":"Redes"}'
Invoke-RestMethod : {"status":"fail","message":"Faltan campos obligatorios (nombre, codigo, creditos)"}
En línea: 1 Carácter: 1
+ Invoke-RestMethod -Method POST -Uri http://localhost:3000/api/cursos  ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> try { Invoke-WebRequest -UseBasicParsing -Method POST -Uri http://localhost:30
00/api/cursos -ContentType "application/json" -Body '{"nombre":"Redes"}' } catch { $_.Exception.Response.StatusCode.value__ }
400
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06>
```

○ PUT /api/cursos/:id: Actualiza las propiedades del curso. Retorna 200 OK o 404 Not Found.

```
37  exports.updateCurso = (req, res) => {
38    const curso = cursos.find(c => c.id === parseInt(req.params.id));
39    if (!curso) {
40      return res.status(404).json({ status: "fail", message: "Curso no encontrado" });
41    }
42    const { nombre, codigo, creditos } = req.body;
43    if (nombre) curso.nombre = nombre;
44    if (codigo) curso.codigo = codigo;
45    if (creditos) curso.creditos = Number(creditos);
46    res.status(200).json({ status: "success", data: curso });
47  };
```

Ejecucion:

```
Type "application/json" -Body '{"creditos":5}'

status  data
------  ----
success @{id=1; nombre=Introducción al Desarrollo Web; codigo=IDWeb; creditos=5}


/cursos/1 -ContentType "application/json" -Body '{"creditos":5}').StatusCode
200
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> Invoke-RestMethod -Method PUT -Uri http://localhost:3000/api/cursos/999 -Conte
ntType "application/json" -Body '{"creditos":5}'
Invoke-RestMethod : {"status":"fail","message":"Curso no encontrado"}
En línea: 1 Carácter: 1
+ Invoke-RestMethod -Method PUT -Uri http://localhost:3000/api/cursos/9 ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> try { Invoke-WebRequest -UseBasicParsing -Method PUT -Uri http://localhost:300
0/api/cursos/999 -ContentType "application/json" -Body '{"creditos":5}' } catch { $_.Exception.Response.StatusCode.value__ }
404
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06>
```

○ DELETE /api/cursos/:id: Elimina el curso del arreglo local y retorna 200 OK con mensaje de confirmación o 204 No Content.

```
// ===== PERSONA 3: Eliminación =====
exports.deleteCurso = (req, res) => {
  const indice = cursos.findIndex(c => c.id === parseInt(req.params.id));
  if (indice === -1) {
    return res.status(404).json({ status: "fail", message: "Curso no encontrado" });
  }
  cursos.splice(indice, 1);
  res.status(200).json({ status: "success", message: "Curso eliminado correctamente" });
};
```

Ejecucion:

```
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> Invoke-RestMethod http://localhost:3000/api/cursos

status  results data
------  ------- ----
success       3 {@{id=1; nombre=Introducción al Desarrollo Web; codigo=IDWeb; creditos=4}, @{id=2; nombre=Calculo I; codigo=CCL1; cr...


PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> (Invoke-WebRequest -UseBasicParsing -Method DELETE http://localhost:3000/api/c
ursos/2).StatusCode
200
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> try { Invoke-WebRequest -UseBasicParsing -Method DELETE http://localhost:3000/
api/cursos/2 } catch { $_.Exception.Response.StatusCode.value__ }
404
PS C:\Users\HP\Documents\IDWeb\LAB7\IDWeb-Laboratorio-06> Invoke-RestMethod http://localhost:3000/api/cursos

status  results data
------  ------- ----
success       2 {@{id=1; nombre=Introducción al Desarrollo Web; codigo=IDWeb; creditos=4}, @{id=3; nombre=Estructuras Discretas II; ...
```

4. Pruebas y Documentación:

○ Probar todos los endpoints utilizando Postman / Thunder Client y exportar la colección de pruebas en formato JSON.

Utilizando Thunder Client

```
> idweb-laboratorio-06@1.0.0 start
> node app.js

API Express en puerto 3000
```

- **Prueba 3:**

```
201
X-Powered-By
Content-Type
Content-Length
ETag
Date
Connection


  "status"  "success"
  "data"
    "id"  4
    "nombre"  "Estructura de Datos"
    "codigo"  "ED102"
    "creditos"  4
```

- **Prueba 4**

```
400
X-Powered-By
Content-Type
Content-Length
ETag
Date
Connection

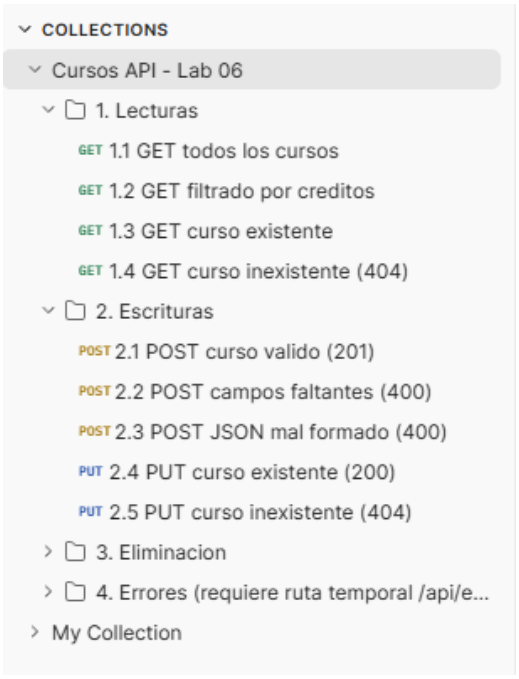

  "status"  "fail"
  "message"  "Faltan campos obligatorios: nombre, codigo, creditos"
```

- **Prueba 5**

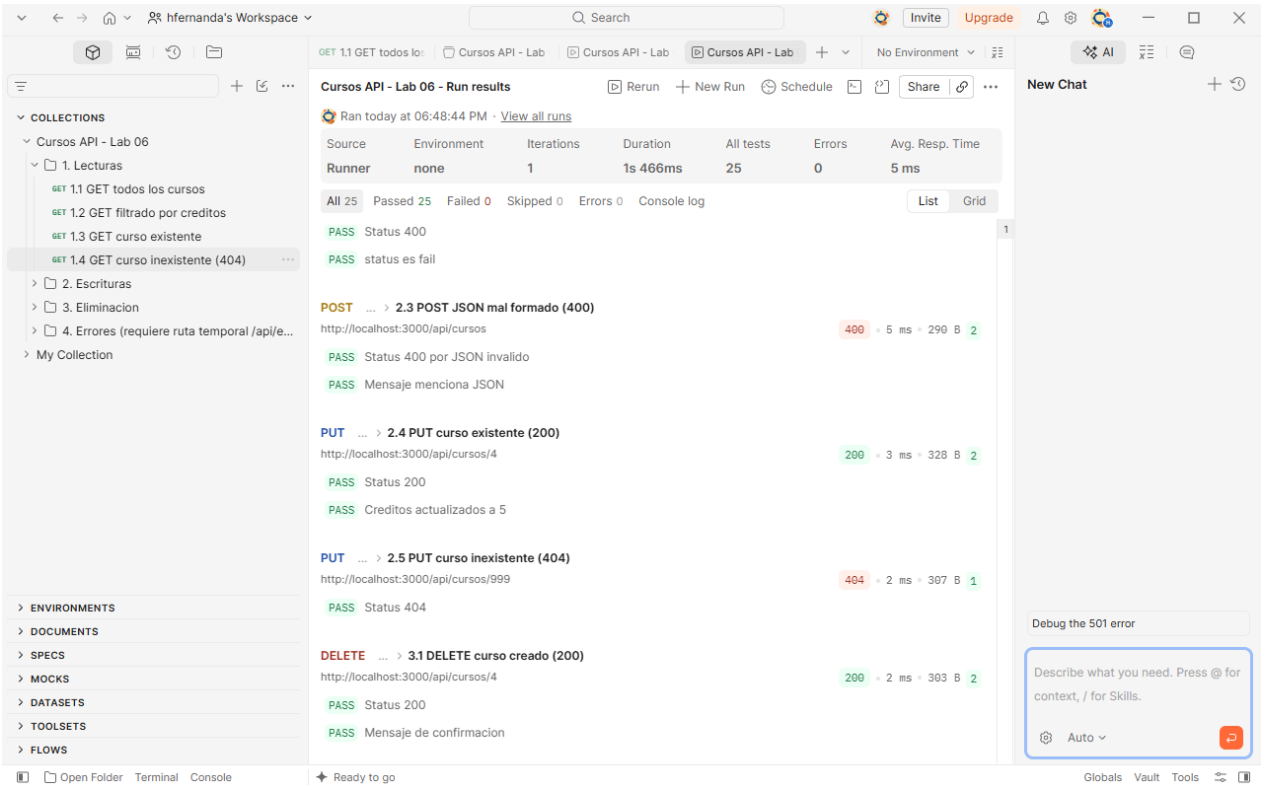
```
200
X-Powered-By
Content-Type
Content-Length
ETag
Date
Connection


  "status"  "success"
  "data"
    "id"  1
    "nombre"  "Curso Actualizado"
    "codigo"  "CUR101"
    "creditos"  5
```

Utilizando Postman



- runner-verde



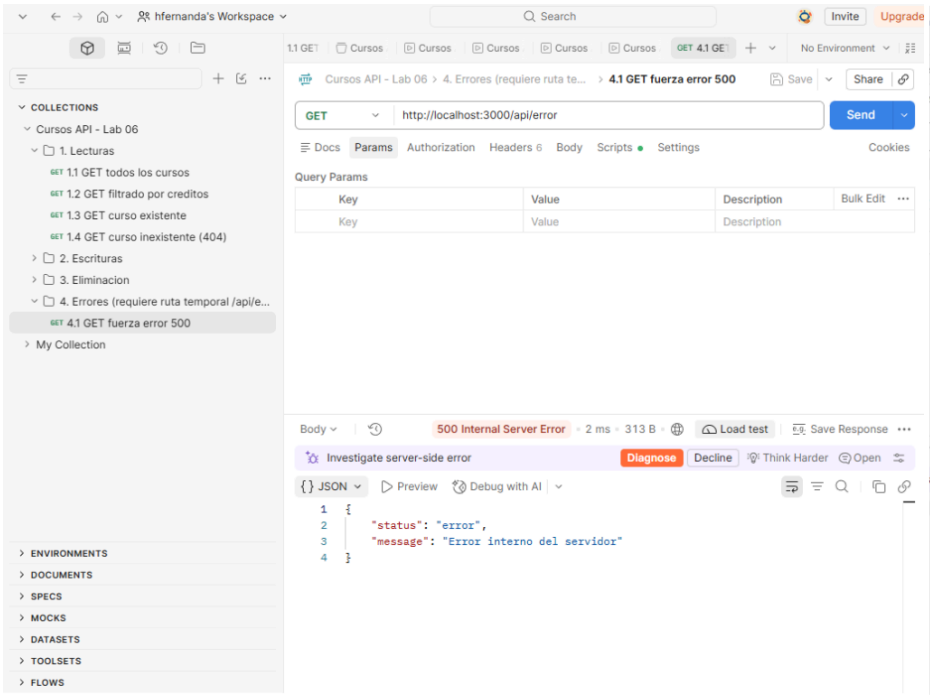
- logger-consola

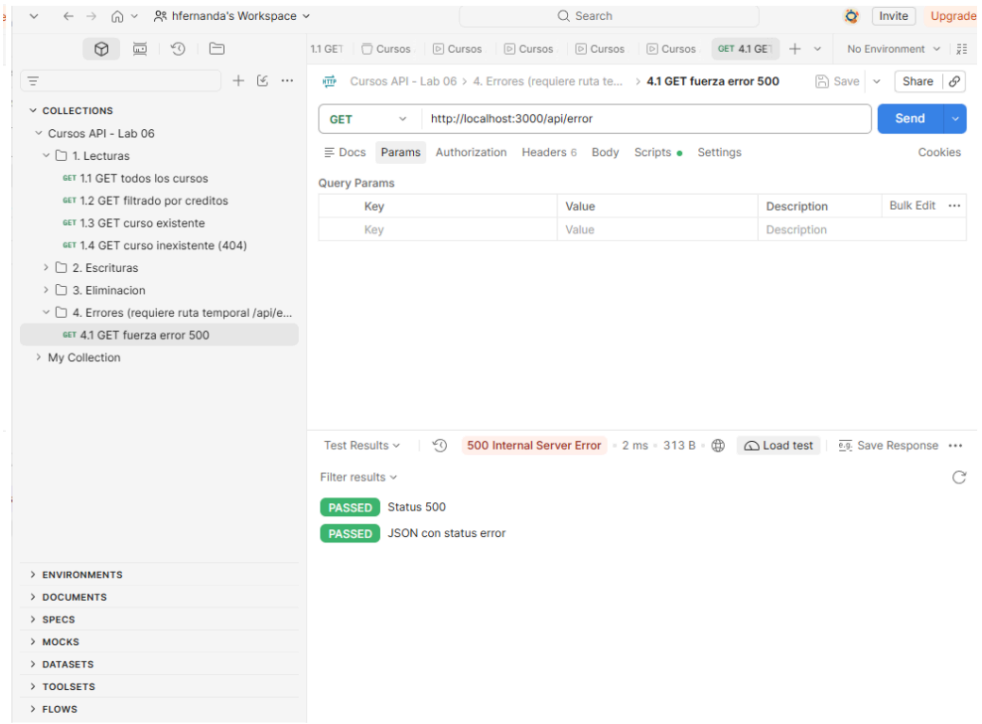
```
API Express en puerto 3000
[2026-10-26T23:48:44.657Z] GET /api/cursos - 200 - 9 ms
[2026-10-26T23:48:44.739Z] GET /api/cursos?creditos=4 - 200 - 0 ms
[2026-10-26T23:48:44.818Z] GET /api/cursos/1 - 200 - 1 ms
[2026-10-26T23:48:44.909Z] GET /api/cursos/999 - 404 - 1 ms
[2026-10-26T23:48:45.019Z] POST /api/cursos - 201 - 1 ms
[2026-10-26T23:48:45.112Z] POST /api/cursos - 400 - 1 ms
SyntaxError: Expected double-quoted property name in JSON at position 17 (line 1 column 18)
    at JSON.parse (<anonymous>)
    at parse (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\body-parser\lib\types\json.js:93:21)
    at D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\body-parser\lib\read.js:194:18
    at AsyncResource.runInAsyncScope (node:async_hooks:227:14)
    at invokeCallback (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\raw-body\index.js:238:16)
    at done (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\raw-body\index.js:227:7)
    at IncomingMessage.onEnd (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\raw-body\index.js:287:7)
    at IncomingMessage.emit (node:events:514:28)
    at endReadableNT (node:internal/streams/readable:1764:12)
    at process.processTicksAndRejections (node:internal/process/task_queues:90:21)
[2026-10-26T23:48:45.302Z] PUT /api/cursos/4 - 200 - 1 ms
[2026-10-26T23:48:45.391Z] PUT /api/cursos/999 - 404 - 0 ms
[2026-10-26T23:48:45.486Z] DELETE /api/cursos/4 - 200 - 1 ms
[2026-10-26T23:48:45.563Z] GET /api/cursos/4 - 404 - 1 ms
[2026-10-26T23:48:45.655Z] DELETE /api/cursos/999 - 404 - 0 ms
```

- prueba-500-consola

```
[2026-10-27T00:11:08.259Z] PUT /api/cursos/999 - 404 - 1 ms
[2026-10-27T00:11:08.354Z] DELETE /api/cursos/4 - 200 - 1 ms
[2026-10-27T00:11:08.432Z] GET /api/cursos/4 - 404 - 0 ms
[2026-10-27T00:11:08.524Z] DELETE /api/cursos/999 - 404 - 0 ms
Error: Error de prueba
    at D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\app.js:11:37
    at Layer.handleRequest (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\router\lib\layer.js:152:17)
    at next (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\router\lib\route.js:157:13)
    at Route.dispatch (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\router\lib\route.js:117:3)
    at handle (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\router\index.js:435:11)
    at Layer.handleRequest (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\router\lib\layer.js:152:17)
    at D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\router\index.js:295:15
    at processParams (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\router\index.js:582:12)
    at next (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\node_modules\router\index.js:291:5)
    at module.exports (D:\Progrmaciontwt\IDWeb\Lab06\IDWeb-Laboratorio-06\middlewares\logger.js:9:3)
[2026-10-27T00:11:08.601Z] GET /api/error - 500 - 0 ms
```

- prueba-500





REPOSITORIO GITHUB: https://github.com/mychemicalheart1712/IDWeb-Laboratorio-06

III. CUESTIONARIO:

1. ***¿Qué es un Middleware en Express y cuál es el impacto de no invocar la función next() dentro de su ejecución?***

   Un middleware es una función con tres parámetros: (req, res, next). Express las encadena y cada petición las atraviesa en orden. Cada una puede leer o modificar la petición o la respuesta (por ejemplo, añadir req.usuario), terminar la petición enviando una respuesta, o pasar el control al siguiente con next().

   Si no se llama a next() ni se envía una respuesta, la petición no avanza: los siguientes middlewares y controladores nunca se ejecutan, y el cliente queda en estado pendiente.

2. ***Explique la diferencia de propósito semántico y de implementación entre req.params, req.query y req.body en Express.***
   - ***req.params:*** *guarda los valores que van dentro de la ruta y sirven para identificar un recurso en particular. Se declaran con `:` en la ruta (por ejemplo /api/cursos/:id) y se leen con req.params.id. Llegan como texto.*

- ***req.query:*** *guarda los valores que van después del ? en la URL. Se usan para filtrar o buscar dentro de una lista (por ejemplo /api/cursos?creditos=4) y se leen con req.query.creditos. No se declaran en la ruta y también llegan como texto.*
- ***req.body:*** *guarda los datos que el cliente envía dentro de la petición, como un JSON o un formulario. Se usa para crear o actualizar información (POST y PUT). No va en la URL, y al inicio está vacío (undefined): solo se llena cuando se usa un middleware como express.json() o express.urlencoded().*

3. ***¿Por qué es fundamental devolver códigos de estado HTTP semánticos (200, 201, 400, 404, 500) en una API RESTful en lugar de responder siempre con código 200 OK?***

   Los códigos de estado HTTP son fundamentales en una API REST porque le indican al cliente qué ocurrió con su petición. Si la API respondiera siempre 200 OK, el cliente tendría que leer cada mensaje para saber si hubo un error, y herramientas como Postman no podrían detectar los fallos por sí solas.

   - ***2xx (éxito):*** *200 OK, 201 Created (al crear un curso) y 204 No Content.*
   - ***4xx (error del cliente):*** *400 Bad Request (faltan campos), 401 Unauthorized y 404 Not Found (el curso no existe).*
   - ***5xx (error del servidor)****: 500 Internal Server Error y 503 Service Unavailable.*

   Enviar el estado HTTP correcto junto con un JSON informativo asegura que la API sea limpia, estándar y fácil de entender para otros desarrolladores

CONCLUSIONES

- *Durante este laboratorio 6 se comprendió acerca de la organización en capas (Routes y Controllers) desacopla los endpoints de la lógica interna, evitando código monolítico y facilitando la mantenibilidad*
- *Del mismo modo la canalización de funciones middleware permite interceptar, transformar y auditar las solicitudes, además de capturar excepciones de forma centralizada sin interrumpir la ejecución del servidor.*
- *El uso preciso de códigos de estado como 200 OK, 201 Created, 400 Bad Request, 404 Not Found y 500 Internal Server Error es fundamental para comunicar el resultado exacto de la operación al cliente de forma estandarizada, evitando enmascarar errores tras un código genérico 200 OK.*
- *Por último la organización de colecciones en herramientas como Postman o Thunder Client con aserciones integradas permite realizar pruebas automatizadas, garantizando que el contrato de la API se cumpla ante cambios en el código.*

METODOLOGÍA DE TRABAJO

1. *Configuración del Entorno y Proyecto*
2. *Estructuración Modular de Capas*
3. *Implementación del Middleware Incorporado*
4. *Desarrollo de Middlewares Personalizados*
5. *Construcción de Controladores (Lógica CRUD)*
6. *Definición de Rutas y Mapeo de Endpoints*
7. *Pruebas e Integración en Entorno de Ejecución*
8. *Documentación, Expresión Semántica y Exportación*

REFERENCIAS Y BIBLIOGRAFÍA

- *Express.js Community. (2025). Express Routing and Middleware Documentation. Expressjs.com.*
- *Restlet Team. (2023). RESTful API Design Best Practices. O'Reilly.*
- *Wilson, M. (2021). Building RESTful Web APIs with Node.js and Express. Packt Publishing.*